EPFL - Computer Science

CS-550 Formal Verification

Lecture 01 — October 11, 2026

Author: Jonathan Dutoit

Professor: Viktor Kuncak

Logistics

Grade breakdown

- 5% first lab (using Stainless)
- 5% second lab (using Lean - can be used to proof AI generated code)
- 30% **in-class exam** solving problems in Lean on computers (late October)
- 40% **in-class paper exam** (November)
- 20% **final group projects and their presentation in class** (last weeks of the semester)
  - Some people verified programs. Additions to automated verification systems. They had the form of case studies. Relevant mathematical theory, etc... Groups are composed of **3 peoples**. All labs and final projects are to be made with the formed group.

Introduction

Formal Verification

Goal: Rigorously prove that computer systems (can be software and hardware, focus on **software**) "do what they should do" = satisfy specification.

How?

1. Define a mathematically rigorous notion of a system satisfying a specification
2. Use combination of automated tools and human effort to construct the proof.

Formal Methods for Computer Systems

> Computer science benefits from the same foundation mechanisms to check rigorous proofs as mathematics. Computer science systems need to prove things about large and messy artifacts. e.g., Theorem: These 100K lines of code have no buffer overflow and behave from outside like this 10k simpler (less efficient) prototype.
>
> Because computer system descriptions are large and theorem's not very deep, there is more emphasis on algorithms to automatically construct proofs (decision procedures, model checking, abstract interpretation) that complete certain specific kinds of proofs. This may be somewhat changing with LLMs, with proof checking becoming more important for computer systems. We will learn how to construct systems to check and find proofs.

Comparing to Testing

Formal verification for proving correctness of computer systems. This is hard work. Why bother? Maybe we can use a **really fast** fuzz tester?

```
assert(x + y == y + x)
```

Suppose we can run 10 tests every **nanosecond**. How long to test **all** cases?

$$\begin{aligned} \Rightarrow 2^{64} \cdot 2^{64} = 2^{128} > 10^{38} \\ \text{Seconds: } 10^{28} \\ \text{Days: } 1.15 \cdot 10^{23} \\ \text{Years: } 3.15 \cdot 10^{20} \end{aligned}$$

Ten billion times since "big bang".

Feasible Alternative: Automated Theorem Proving

A modern software verifier has **built-in** knowledge of commutativity.

- it's a mathematical **theorem** about integers module 2^{64}
- we can also prove theorems about unbounded integers (need infinitely many tests)

A verifier also users **logical rules**, studies in **formal (mathematical) logic**, to take existing theorems, like $x + y = y + x$, and derive new ones, like $x + (a + 1) = (a + 1) + x$.

Verifiers also use **automated theorem proving procedures**, which can discover an infinite number of facts using know theorems and rules.

Different kinds of proving procedures, depending on success guarantees:

- **Decision procedures** (linear arithmetic, SAT, bitvectors), proof checkers: algorithms with upper bounds on time by which they terminate with yes/no answer
- **semi-decision procedures** (first-order logic proves): no upper bound, but for every theorem there exists a time by which they discover that it hold (complete)
- **heuristics** (simplifiers, induction heuristics, type inference): even if formula is valid, they may run forever, or say "don't know"

Compiling to Formulas (example: Stainless verifier)

How do we go from program correctness statements to theorems?

Compile programs and properties to formulas! We call these formulas **verification conditions**. If verification is valid formula, then program satisfies specification. Analogy:

programming language compiler	verification-condition generator
program $\rightarrow$ machine code	(program + specification) $\rightarrow$ formula

$$\boxed{\text{program verifier}} = \boxed{\text{verification condition generator}} \oplus \boxed{\text{theorem prover}}$$

Modeling and Verification of Discrete-Time Systems

General models (rule systems, lambda calculus, Turing machines) usually operate step by step $t \in N : s_{t+1} = F(s_t)$. Hardware designers decided to use discrete time steps (GHz clock rate).

Definition. Transition system as (S, I, r, A):

- S - set containing all states of the system. S finite $\Rightarrow$ *finite-state system*
- $I \subseteq S$ - set of possible initial states of the system.
- $r \subseteq S \times A \times S$ - transition relation; $(s, a, s') \in r$ means: with env. signal a, system can move in one step from state s to s'
- A - set of signals with which the system communicates with the environment.
- (Optional) $E \subseteq S$ - set of error states.
- (Optional) $G \subseteq S$ - set of good states.

Analogous FSM Tuple Notation

In standard computer science notation, an FSM is defined as the tuple:

$$M = (Q, \Sigma, \delta, Q_0, F).$$

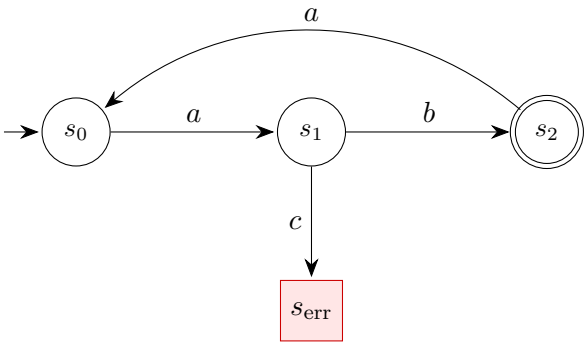
Transition System	**Analogous FSM**	**Description & Equivalence**
S (Set of states)	Q	Set of all machine states (S is finite $\iff$ Q is finite)
A (Environment signals)	Σ	Input Alphabet (set of input symbols/events)
$r \subseteq S \times A \times S$	$\delta \subseteq Q \times \Sigma \times Q$	Transition relation
$I \subseteq S$	$Q_0 \subseteq Q$	Set of initial states
$G \subseteq S$	$F \subseteq Q$	Set of accepting / good states
$E \subseteq S$	$F_{\text{err}} \subseteq Q$ (or $Q \setminus F$)	Set of error / trap states

Relation vs. Function:

- Because $r \subseteq S \times A \times S$ is a **relation** (not a function), a state s receiving signal a can transition into **zero**, **one**, or **multiple** potential next states s'.

- This makes the system inherently **non-deterministic** (analogous to an NFA or Labeled Transition System).
- For a **Deterministic Finite Automaton (DFA)**, r is restricted to a partial function $\delta : Q \times \Sigma \to Q$.

Illustration



$$M = (S, I, r, A, E, G)$$
$$I = \{s_0\}, \quad G = \{s_2\}, \quad E = \{s_{\text{err}}\}$$
$$A = \{a, b, c\}$$

Figure 1: Transition System M as a State Transition Graph

Definition. A Trace (Trajectory) of the System *M = (S, I, r, A)* is a finite or infinite sequence $s_o, a_0, s_1, a_1, s_2, ...$ starting from $s_o \in I$ with steps given by r:

$$\boxed{s_0} \qquad s_0 \in I$$
$$\downarrow a_0 \qquad a_0 \in A$$
$$\boxed{s_1} \qquad (s_0, a_0, s_1) \in r$$
$$\downarrow a_1 \qquad a_1 \in A$$
$$\boxed{s_2} \qquad (s_1, a_1, s_2) \in r$$
$$\dots$$

In general, we require $(s_i, a_i, s_{i+1}) \in r$ for all i in the length of the sequence. If the trace is finite, we assume it ends with a state s_n and call n its length.

Traces(M) is the set of all traces of *M*.

Reach

Definition. *Reach(M)*: reachable states s_n for which there exists a trace that ends in s_n.

$$\text{Reach(M)} = \{s_n \mid \exists \mathbf{n}.\exists(s_o, a_o, s_1, a_1, ..., s_n) \in \text{Traces(M)}\}.$$

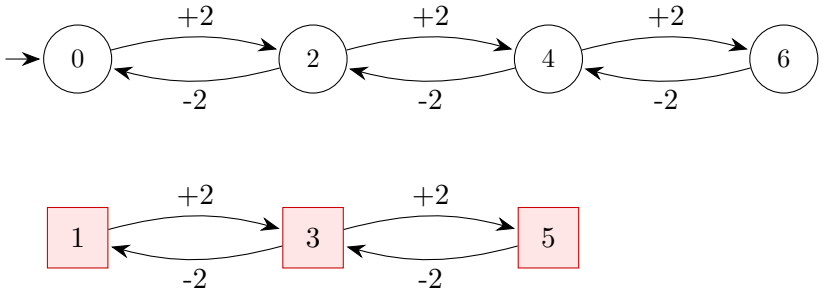
Explicit-State Reachability Checking

The goal is to determine whether an **error state** E is reachable from any initial state I in a finite-state transition system $M = (S, I, r, A)$.

- Input: $M = (S, I, r, A)$, where S is **finite**, $E \subseteq S$ (error states)
- Output: either a $(s_0, a_0, s_1, a_1, ..., s_n) \in \textit{Traces(M)}$ where $s_n \in E$, or the answer "Safe" if no such trace exists.
- Idea: graph reachability from nodes in I, following edges in $(s, a, s') \in r$ as long as we have not seen s' before.
- To be able to report the trace, build a directed reachability graph of explored edges (never create cycles or duplicate nodes)
- If no edge in r leads to a previously unexplored node, we stop (this must eventually happen because S is finite).
- **Graph search implementation**: graph reachability using a work list. For efficiency, define three sets of nodes in a graph. Key operation: from frontier node s, add all of it's unexplored non-frontier successors to the frontier, move s to explored.
  - Set of all nodes
  - Explored nodes: whose all successors we have explored
  - Frontier nodes (worklist): we have explore them but not their successors.

Exercise 1: Bounded Counter

Consider a system with $S = \{0, 1, 2, ..., 6\}$ that takes signals $A = \{+, -\}$ with initial state 0 and count up by 2 on + and down by 2 on - but never goes below 0 or above 6 (stays in the state if needed). Write down the transition system definition and prove that the state $E = \{3\}$ is not reachable using explicit-state reachability algorithm. Draw the reachability graph.



$$M = (S, I, r, A, E)$$
$$r = \{(0, +, 2),\ (2, +, 4),\ (4, +, 6),\ (2, -, 0),\ (4, -, 2),\ (6, -, 4),\ (1, +, 3),\ (3, +, 5),\ (3, -, 1),\ (5, -, 3)\}$$

Figure 2: $Reach(M) = \{0, 2, 4, 6\}$ *and* $E = \{1, 3, 5\} \Rightarrow Reach(M) \cap E = \emptyset$

Simplified Transition Relation and Reachable States

Idea: ignore the environment signals defined by a. Hence, we consider all possible transitions from s to s', keeping only the connection between states:

Let $M = (S, I, r, A)$ be a transition system.
Definition. $\bar{r} = \{(s, s') \mid \exists a \in A.(s, a, s') \in r\}$

Note: even if r is deterministic, $\bar{r}$ can become non-deterministic. *Reminder*: a relation is non-deterministic when a state has multiple possible next states. Removing action labels can combine transitions that were distinguished by their actions.

Counter ex.: $\bar{r} = \{(0, 2), (2, 4), (2, 0), (4, 6), (4, 2), (6, 4)\}$

Composition of relations: $r_1 \circ r_2 = \{(x,z) \mid \exists y.(x,y) \in r_1 \wedge (y,z) \in r_2\}$
Iteration (paths of length n): $r_1^0 = \Delta = \{(x,x) \mid x \in A\}, \quad r_1^{n+1} = r_1 \circ r_1^n$

- $r^0 = \Delta$: paths of length zero. Every state reaches itself without taking a step.
- $r^1 = r$ paths of one step.
- r^n: paths of exactly n steps.
- r^*: paths of any finite length, including zero.

Transitive closure of r_1:

$$r_1^* = \bigcup_{n \geq 0} r_1^n \quad \text{relates endpoints of all finite paths in graph given by } r_1.$$

Image of a set under relation: $r_1[X] = \{y \mid \exists x \in X.(x,y) \in r_1\}$. This simply means: **find every state that can be reached in one step from a state in X**. *Counter ex.:* $\bar{r}[X] = \{0,2,4\}$ since from 0 we can reach 2, from 2 we can reach 0 and 4.

Theorem

Reach(M) $= (\bar{r}) * [I]$ (*end points of all finite paths starting in I).* Intuitively, this means that *Reach(M)* are the states collected by all arbitrary long paths where pairs are precisely the image of the set I under the transitive closure.

We can also define *post(X)* $= \bar{r}[X]$ if $X \subseteq S$ under the system $M = (S, I, r, A)$. It means **all immediate successors of the states in X**.

$\Rightarrow \text{post}^0(X) = X$, $\text{post}^{n+1}(X) = \text{post}(\text{post}^n(X))$

Counter ex.: starting from $I = \{0\}$:

- $\text{post}^0(I) \Rightarrow \{0\}$ *Initial states*
- $\text{post}^1(I) \Rightarrow \{2\}$ *Reachable in one step*
- $\text{post}^2(I) \Rightarrow \{0,4\}$ *Reachable in two steps*
- $\text{post}^3(I) \Rightarrow \{2,6\}$ *Reachable in three steps*

Theorem

$$\bigcup_{n \geq 0} \text{post}^n(I) = \text{Reach(M)}.$$

Invariant

Def. Invariant P of the system M is any superset of reachable states: $Reach(M) \subseteq P$.

- P is a property satisfied by all reachable states (though not all states in P need to be reachable).
- In every trace, by definition $s_i \in \text{Reach(M)} \subseteq P$. So the property $s_i \in P$ remains in-variant (does not change) as the system makes a step from i to $i + 1$.

Example: the counter is always even. Consider $P = \{s \in S \mid \text{s is even}\}$. We found that *Reach(M)* $= \{0, 2, 4, 6\}$ and we have that $P = \{0, 2, 4, 6\}$. Since every reachable state is in

P, this is an invariant. Note that $P = \{0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6\} = S$ is also an invariant because all reachable states are in S.

Inductive invariant

This is one of the most important concepts in formal verification because it lets us prove properties without enumerating every reachable state.

> **Def. Inductive invariant** *Ind* is a set of states Ind $\subseteq S$ that satisfies the following:
> - $I \subseteq$ Ind (holds initially)
> - if $s \in$ Ind and $(s, a, s') \in r$, then $s' \in$ Ind.
>
> In simple terms: start inside the set and never leave the set. If both conditions hold, every reachable state must belong to *Ind*.

Example: proving the counter is always even. Choose $Ind = \{0,\ 2,\ 4,\ 6\}$. Let's verify both conditions:

- **Condition 1 - Initial states**
  - The initial state is 0, and $0 \in$ Ind. Therefore, $I \subseteq$ Ind holds.
- **Condition 2 - Transitions**.
  Check every transition from a state in *Ind*:
  - $0 \to 2$: remains in *Ind*
  - $2 \to 0$ and $2 \to 4$: remain in *Ind*
  - $4 \to 2$ and $4 \to 6$: remain in *Ind*
  - $6 \to 4$: remains in *Ind*.

  Therefore, no transition leaves *Ind*.

We proved the result by checking the initial state and the transitions, rather than exploring every possible trace.